PROYECTO 1 - PROBLEMAS NP Y DP

Análisis y diseño 2

Universidad del Valle – Sede Tuluá

Julio Cardona - 2359654

Jose David Mármol - 2266370

Alejandra Osorio Giraldo - 2266128

Alejandro Garzón Mayorga - 2266088

Andrés Narváez - 2259545

Prof. Miguel Angel Askar Rodriguez

4 / 12 / 2026

1. Analisis teorico del problema

- **Descripción formal del problema asignado**
  El problema de asignación de presupuesto limitado consiste en seleccionar un conjunto de proyectos a financiar a partir de un presupuesto total disponible, logrando maximizar el retorno total obtenido sin exceder el presupuesto asignado.
  tenemos:
  - Un conjunto de n proyectos: P={1,2,3,…,n}
  - Cada proyecto tiene asociado un costo: $c_i>0$
  - un retorno esperado: $r_i>0$
  - Un presupuesto total disponible B>0
  - Variables de decisión:
    x_i={ 1 si el proyecto i es seleccionado, 0 si el proyecto i no es seleccionado}

  restricciones del problema
  - El costo total de los proyectos no debe exceder el presupuesto disponible
    $\sum(i=1 \text{ hasta } n)\ c_i\ x_i \leq B$

  Función objetivo
  - El objetivo es maximizar el retorno total obtenido por los proyectos seleccionados
    $MAX\sum(i=1 \text{ hasta } n)\ r_i\ x_i$
- **Justificación de por que el problema pertenece a la clase NP**
  Para cada proyecto tenemos dos opciones, seleccionarlo 1 o no seleccionarlo 0
  Esto significa que toma un tiempo exponencial de 2^n.
  También podemos verificar una solución en tiempo polinomial, Dado un conjunto de proyectos, podemos calcular la suma de los costos de los proyectos seleccionados y verificar que no excedan el presupuesto disponible, esta tarea de sumar se hace en tiempo O(n), también el cálculo del retorno total se hace en tiempo O(n). Lo que demuestra que el problema pertenece a NP.
- **Análisis de su complejidad computacional**
  El problema de asignación de presupuesto limitado tiene una alta complejidad computacional por el número de combinaciones de proyectos que pueden seleccionarse. Como habíamos analizado antes cada proyecto tiene dos posibles decisiones, ser seleccionado o no ser seleccionado, por lo tanto el número total de combinaciones de selección de proyectos es de 2^n. Para encontrar la mejor solución posible y maximizar el retorno sin exceder el presupuesto debemos evaluar cada combinación realizada en el peor de los casos. El problema presenta una complejidad exponencial.

2. Propuesta de solución mediante Programación Dinámica

El problema de asignación de presupuesto limitado presenta subestructura óptima, ya que la solución óptima para un conjunto de proyectos depende de las soluciones óptimas de subconjuntos más pequeños. Por esta razón, es posible abordarlo mediante programación dinámica.

Se define una función dp[i][b] como el máximo retorno que se puede obtener utilizando los primeros i proyectos con un presupuesto disponible b.

Casos base

Si no se consideran proyectos, el retorno máximo es cero para cualquier presupuesto:

$$dp[0][b] = 0$$

De igual manera, si el presupuesto disponible es cero, no es posible seleccionar ningún proyecto:

$$dp[i][0] = 0$$

Relación de recurrencia

Para cada proyecto i, se evalúan dos posibles decisiones:

Si el costo del proyecto c[i] es mayor que el presupuesto disponible b, no es posible incluirlo, por lo tanto:

$$dp[i][b] = dp[i-1][b]$$

Si el proyecto puede ser incluido (c[i] <= b), se tienen dos opciones: no incluirlo o incluirlo. En este caso se toma el máximo entre ambas:

$$dp[i][b] = \max(dp[i-1][b], r[i] + dp[i-1][b - c[i]])$$

Este proceso se repite para todos los proyectos y todos los valores posibles de presupuesto hasta completar la tabla.

Resultado final

La solución óptima del problema se obtiene en:

$$dp[n][B]$$

donde n es el número total de proyectos y B es el presupuesto disponible.

Análisis de complejidad

La complejidad temporal del algoritmo es O(n * B), ya que se llena una tabla de tamaño n x B.

La complejidad espacial también es O(n * B), debido al almacenamiento de la tabla de programación dinámica.

3. Propuesta de solución mediante un enfoque Voraz:

Definición del criterio de selección:

Para este enfoque, se utiliza el criterio de relación retorno-costo (Pi), seleccionando en cada paso el proyecto que ofrezca mayor rentabilidad por unidad de presupuesto:

$$\rho_{i=\frac{r_1}{c_1}}$$

Donde Ri es el retorno y Ci el costo del proyecto i. La estrategia consiste en elegir siempre el proyecto con el mayor ratio disponible que no exceda el presupuesto restante.

Diseño del algoritmo

El proceso se ejecuta en tres etapas:

1. **Cálculo:** Se obtiene el ratio Pi para cada proyecto del conjunto P.
2. **Ordenamiento:** Se organiza el conjunto en orden descendente (P1$\geq P2 \geq \ldots \geq Pn$)
3. **Selección:** Se recorre la lista y se selecciona cada proyecto (Xi= 1) solo si su costo Ci es menor o igual al presupuesto disponible actual.

Justificación y Complejidad

Este diseño prioriza la eficiencia inmediata, reduciendo el espacio de búsqueda a una sola pasada sobre una lista priorizada.

- **Complejidad Temporal:** Dominada por el ordenamiento, siendo **O(n log n)**.
- **Complejidad Espacial: O(n)** para almacenar los datos de los proyectos y el vector de resultados.

Discusión sobre optimalidad A diferencia de la programación dinámica, el enfoque voraz no garantiza el óptimo en problemas de tipo 0/1. Esto se debe a que el problema carece de la propiedad de elección voraz; es decir, una secuencia de decisiones óptimas locales no asegura alcanzar una solución óptima global. Al operar bajo un esquema de selección inmediata e irrevocable, el algoritmo puede comprometer el presupuesto con proyectos de alto ratio individual, pero generar un remanente inutilizado (espacio desperdiciado) que una combinación distinta de proyectos podría haber aprovechado para maximizar el retorno total.

Ejemplo de fallo (B = 50):

Proyecto	Costo (ci)	Retorno (ri)	Ratio (pi)
1	20	100	5.0
2	10	60	6.0
3	30	120	4.0

Solución Voraz: Selecciona proyectos 2 y 1. **Retorno: 160**. (Sobra 20 de presupuesto).

Solución Óptima: Selecciona proyectos 1 y 3. **Retorno: 220**. (Ajuste exacto al presupuesto).

Comparación con Programación Dinámica:

Característica	Enfoque Voraz	Programación Dinámica
Tipo de decisión	Local e inmediata	Global y exhaustiva
Complejidad	O(n log n)	O(n * B)
Garantía de óptimo	No	Sí
Uso de memoria	Bajo (O(n))	Alto (O(n * B))

Conclusión de la estrategia

El enfoque voraz es ideal por su rapidez y escalabilidad en escenarios con presupuestos masivos donde la programación dinámica es inviable, a pesar de sacrificar la garantía del resultado óptimo global.

4. Implementación

La implementación del proyecto se desarrolló en Python y consta de dos enfoques para resolver el problema de asignación de presupuesto limitado. Por un lado, se implementó la solución mediante Programación Dinámica, la cual construye una tabla de estados para calcular el retorno máximo posible con los primeros proyectos y un presupuesto dado. Esta solución permite obtener el valor óptimo y reconstruir los proyectos seleccionados. Por otro lado, se implementó un enfoque Voraz basado en la relación retorno-costo, que ordena los proyectos por rentabilidad unitaria y selecciona aquellos que caben en el presupuesto restante.

Para facilitar la comparación, ambas soluciones se ejecutan sobre los mismos conjuntos de entrada y reportan el valor total obtenido, los índices de los proyectos elegidos y el costo total utilizado. La implementación incluye validación de entradas, pruebas unitarias y un

programa principal que ejecuta varios casos de ejemplo, incluyendo un contraejemplo donde el enfoque Voraz no alcanza la solución óptima.

Github URL: https://github.com/chronischer-zauderer/Proyecto-1-ADA-II

5. Pruebas
Se realizaron pruebas unitarias para verificar el comportamiento correcto de ambas soluciones. En los casos pequeños, la Programación Dinámica y el enfoque Voraz pueden coincidir en la misma solución; sin embargo, en el contraejemplo diseñado para el proyecto se observa que el enfoque Voraz no siempre produce el óptimo global, mientras que la Programación Dinámica sí lo garantiza.

Además, se ejecutaron pruebas comparativas con diferentes tamaños de entrada para analizar el comportamiento temporal de ambos métodos. Los resultados muestran que el enfoque Voraz es considerablemente más rápido, mientras que la Programación Dinámica ofrece mejores resultados en términos de optimalidad. Por tanto, las pruebas confirman la diferencia entre eficiencia computacional y garantía de optimalidad.

6. Análisis del proceso

Camino de cómo hallar los modelos

El desarrollo del proyecto inició con la identificación de la estructura del problema, reconociendo que la asignación de presupuesto limitado puede representarse como un problema de optimización combinatoria. A partir de esto, se estableció una correspondencia directa con el problema clásico de la mochila 0/1, donde cada proyecto equivale a un objeto con costo (peso) y retorno (valor).

Con esta analogía, se definieron formalmente:

- Las variables de decisión binarias
- La restricción de presupuesto
- La función objetivo de maximización

Posteriormente, se identificó que el problema presenta subestructura óptima y superposición de subproblemas, lo cual permitió justificar el uso de programación dinámica como estrategia de solución exacta.

En paralelo, se exploró un enfoque alternativo basado en heurísticas voraces, motivado por la necesidad de encontrar soluciones más eficientes en términos de tiempo computacional.

Dificultades encontradas

Identificación del modelo correcto:

Inicialmente, no era evidente que el problema correspondiera a una variación directa de la mochila 0/1, lo que dificultó la formulación inicial.

Manejo de la complejidad:

Se evidenció que, aunque la programación dinámica es correcta, su uso puede ser limitado en escenarios con presupuestos muy grandes debido al consumo de memoria y tiempo.

Decisiones de diseño relevantes

Priorización de claridad sobre optimización extrema:
En la implementación se privilegió la comprensión del algoritmo (por ejemplo, usando una tabla completa en programación dinámica) sobre optimizaciones más avanzadas de memoria.

Construcción explícita de la tabla DP:
Se optó por implementar la programación dinámica utilizando una matriz bidimensional dp[n+1][B+1], en lugar de una versión optimizada en espacio.
Esto se hizo con el objetivo de:

facilitar la comprensión del algoritmo
permitir la reconstrucción de la solución óptima (backtracking)
hacer más claro el análisis académico del método

Separación modular de la implementación:

Se dividió la implementación en:

dp.py
greedy.py
main.py
pruebas unitarias

Esta decisión mejora:

mantenibilidad
reutilización
claridad del proyecto

7. Conclusiones

Este proyecto nos permitió entender que un problema aparentemente simple, como asignar un presupuesto, puede volverse complejo cuando se analiza desde el punto de vista algorítmico. Modelarlo correctamente fue fundamental para poder aplicar las distintas estrategias y comprender realmente qué se estaba optimizando.

También aprendimos que este problema pertenece a la clase NP, lo que implica que encontrar la mejor solución puede ser costoso debido a la gran cantidad de combinaciones posibles, aunque verificar una solución sea rápido. Esto explica por qué no es viable usar métodos exhaustivos en la práctica.

Al comparar los enfoques, se evidenció claramente la diferencia entre precisión y eficiencia. La programación dinámica garantiza la solución óptima, pero con un mayor costo computacional, mientras que el enfoque voraz es más rápido, aunque no siempre produce el mejor resultado, como se comprobó en las pruebas.

En general, el proyecto nos deja como aprendizaje principal que no existe un algoritmo ideal para todos los casos, sino que la elección depende del contexto y de los recursos disponibles. Saber cuándo priorizar exactitud o eficiencia es clave en el diseño de soluciones reales.